# Lecture 25 — Lazy Propagation & Range-Update/Range-Query

> Year Curriculum · Prerequisites: Week 23 (segment trees), Week 24 (Fenwick trees), Week 22 (Euler tour) · Estimated time: 2.5 hours

## 1. Overview

The segment tree of Week 23 supports point updates and range queries. Many problems instead update **whole ranges**: "add x to every element in `[l, r]`", "set every element in `[l, r]` to x", "flip all bits in `[l, r]`". Performing a range update as r − l + 1 point updates costs O(n log n) per operation, which is hopeless for 2·10^5 operations.

**Lazy propagation** fixes this. A range update, like a range query, decomposes into O(log n) canonical nodes. At each such node we update the stored aggregate immediately and attach a **lazy tag** saying "my children have not received this update yet". The tag is pushed one level down only when a later operation needs to visit the children. Every operation stays O(log n).

The mechanics (apply/push/pull) are short; the real skill is designing the **tag** so that several pending updates can be composed into one tag of the same shape. This lecture builds that skill with range add, range assign, assign + add together, and affine maps.

## 2. Mechanics

We use the recursive segment tree (node `x` covers `[l, r]`, children `2x` and `2x+1`, size `4n`). Each node stores:

- `val[x]` — the aggregate (sum, min, …) of its segment, **already including** every update that has reached `x`;
- `tag[x]` — the composition of updates that reached `x` but have **not** been passed to its children.

Three helpers:

- `apply(x, l, r, u)` applies update `u` to the whole segment of `x`: it changes `val[x]` (in O(1), using only `val[x]` and the segment length) and composes `u` into `tag[x]`.
- `push(x, l, r)` passes `tag[x]` to both children with `apply` and resets `tag[x]` to the identity.
- `pull(x)` recomputes `val[x]` from the children: `val[x] = combine(val[2x], val[2x+1])`.

The range update is a range query with side effects:

```
update(x, [l, r], [ql, qr], u):
    if disjoint: return
    if covered:  apply(x, l, r, u); return        // stop here!
    push(x, l, r)                                 // children must be up to date
    update(left child ...); update(right child ...)
    pull(x)
```

The range query must also `push` before descending, because children may be stale.

**Invariant.** For every node, `val[x]` equals the true aggregate of its segment after all updates that have been applied to `x` or to an ancestor and pushed down to `x`. Updates still sitting in tags of strict ancestors are not yet reflected — but every operation that reaches `x` passes through those ancestors and pushes them first. Hence whenever we read `val[x]`, it is correct.

**Complexity.** The set of visited nodes is exactly the set visited by a range query: at most 4 per level. `apply`, `push` and `pull` are O(1) (for the tags in this lecture). So updates and queries take O(log n), and memory is 4n values plus 4n tags.

### 2.1 Example: add 5 on [2, 7] with n = 8

The canonical nodes are `[2,3]` and `[4,7]`. At `[2,3]` we do `sum += 5·2`, `tag += 5`; at `[4,7]`, `sum += 5·4`, `tag += 5`. On the way back we pull `[0,3]` and the root. Nodes below `[4,7]` are untouched until some later operation needs them, at which point `push` hands them `+5` each.

## 3. Range add

**Range add, range sum.** Tag = pending addition `a` (identity 0). `apply`: `sum += a·len`, `tag += a`. Composition: two additions add up. Note the factor `len`: the sum of a segment grows by `a` per element.

**Range add, range min (or max).** `apply`: `mn += a`, `tag += a`. No length needed, since adding a constant shifts the minimum by that constant. Codeforces 52C *Circular RMQ* is this, with circular ranges split into two.

If only "range add + range sum" is needed, the two-BIT trick from Week 24 is shorter. Lazy trees become necessary when the query is min/max, or when updates are not additions.

## 4. Range assign

Tag = "set every element to `s`", or **none**. You need a way to represent "none": a boolean flag, or a sentinel value that cannot occur in the input (careful: if 0 is a valid value, 0 cannot be the sentinel). `apply`: `sum = s·len`, tag = `s` (a newer assignment simply overwrites an older pending one). For a min tree, `mn = s`.

## 5. Composing different updates: assign + add

CSES 1735 *Range Updates and Sums* mixes "add x on `[a, b]`" and "set to x on `[a, b]`". The difficulty: a node may have a pending tag when a new update arrives, and the tag must describe the **combined** effect in O(1) space. Look at all orders:

| Pending (older) | New | Combined effect |
|---|---|---|
| add a | add b | add a + b |
| add a | set s | set s (the add is irrelevant) |
| set s | add a | set s, then add a |
| set s | set t | set t |

The combination "set s, then add a" is not a pure set nor a pure add, so the tag type must be the **normal form** `(optional set s) then (add a)`. Check that this form is closed: applying "add b" to `(set s, add a)` gives `(set s, add a + b)`; applying "set t" gives `(set t, add 0)`. Both are in normal form. So:

```
applyTag(node, len, t):
    if t has a set s:  sum = s·len;  tag = (set s, add 0)
    if t.add != 0:     sum += t.add·len;  tag.add += t.add
```

Because `push` passes a whole normal-form tag `(set s, add a)` to a child, `applyTag` processes the set part first and the add part second — the same order the tag represents. Getting this order wrong is the classic bug.

Trace on the CSES sample `a = 2 3 1 1 5 3`: the sum on `[3,5]` is 7; after "add 2 on [2,4]" it is 3 + 3 + 5 = 11; after "set 5 on [2,4]" it is 5 + 5 + 5 = 15.

## 6. The general pattern: monoid acted on by tags

Abstractly, a lazy segment tree needs:

1. A set of node values with an associative `combine` (a monoid).
2. A set of updates (tags) with a composition `compose(new, old)` that is associative and has an identity.
3. An action `apply(tag, value)` computable in O(1) that **distributes** over `combine`: `apply(t, combine(x, y)) = combine(apply(t, x), apply(t, y))`.

Condition 3 is what makes it legal to update a parent without touching its children. Sometimes the node value must be enriched to make it hold — for example a sum-tree with "add" needs the segment length, which you can either pass as a parameter or store in the node as `(sum, len)`.

**Affine updates.** Updates of the form `x → b·x + c` on every element, queries for the sum (mod p). Action: `sum → b·sum + c·len`. Composition of old `(b1, c1)` with new `(b2, c2)`: `x → b2(b1x + c1) + c2`, i.e., `(b2·b1, b2·c1 + c2)`. This single tag type covers add (`b = 1`), multiply (`c = 0`) and assign (`b = 0`). The Library Checker problem *Range Affine Range Sum* (https://judge.yosupo.jp/problem/range_affine_range_sum) is the standard benchmark.

**Arithmetic progressions (CSES 1736 *Polynomial Queries*).** Update: add 1 to `a[l]`, 2 to `a[l+1]`, .... A pending tag is "add an arithmetic progression with first term f and difference d" relative to the node's left end. Two progressions compose by adding firsts and differences. When pushing to the right child, its first term is shifted by `d·(length of the left child)`. Applying to a sum: `sum += f·len + d·len·(len−1)/2`.

**Bitwise updates (Codeforces 242E *XOR on Segment*).** Update: xor every element with x; query: sum. Store per node the count of ones in each of the ~20 bit positions; the tag is an xor mask; applying it flips `cnt[b] → len − cnt[b]` for every bit b set in the mask; composition is xor.

## 7. Lazy propagation on trees

Combine with the Euler tour of Week 22: a subtree is the contiguous range `[tin[v], tout[v]]`. "Recolor the whole subtree of v with color c, and count distinct colors in the subtree of u" (Codeforces 620E *New Year Tree*, at most 60 colors) becomes a range-assign tree whose node value is a 64-bit **mask** of colors present, combined with OR; the answer is `popcount`. Path updates need heavy-light decomposition (Nationals track).

## 8. Reference implementations

### 8.1 Range add + range sum

Input: `n q`, the array, then queries `1 l r x` (add) or `2 l r` (sum). Tested against a brute force on 400 random cases.

```
#include <bits/stdc++.h>
using namespace std;

// Lazy segment tree: range add, range sum.
// Input: n q, array, then queries "1 l r x" (add x on [l,r]) or "2 l r" (sum).
struct LazySeg {
    int n;
    vector<long long> sum, lz;
    LazySeg(const vector<long long>& a) : n(a.size()), sum(4 * a.size()), lz(4 *
a.size(), 0) {
        build(1, 0, n - 1, a);
    }
    void build(int x, int l, int r, const vector<long long>& a) {
        if (l == r) { sum[x] = a[l]; return; }
```

```cpp
        int m = (l + r) / 2;
        build(2 * x, l, m, a);
        build(2 * x + 1, m + 1, r, a);
        sum[x] = sum[2 * x] + sum[2 * x + 1];
    }
    void apply(int x, int l, int r, long long v) {   // add v to every element of node x
        sum[x] += v * (r - l + 1);
        lz[x] += v;
    }
    void push(int x, int l, int r) {
        if (lz[x] != 0) {
            int m = (l + r) / 2;
            apply(2 * x, l, m, lz[x]);
            apply(2 * x + 1, m + 1, r, lz[x]);
            lz[x] = 0;
        }
    }
    void update(int x, int l, int r, int ql, int qr, long long v) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { apply(x, l, r, v); return; }
        push(x, l, r);
        int m = (l + r) / 2;
        update(2 * x, l, m, ql, qr, v);
        update(2 * x + 1, m + 1, r, ql, qr, v);
        sum[x] = sum[2 * x] + sum[2 * x + 1];
    }
    long long query(int x, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return sum[x];
        push(x, l, r);
        int m = (l + r) / 2;
        return query(2 * x, l, m, ql, qr) + query(2 * x + 1, m + 1, r, ql, qr);
    }
    void update(int l, int r, long long v) { update(1, 0, n - 1, l, r, v); }
    long long query(int l, int r) { return query(1, 0, n - 1, l, r); }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto& v : a) cin >> v;
    LazySeg st(a);
    while (q--) {
        int type, l, r;
        cin >> type >> l >> r;
        if (type == 1) {
            long long x;
            cin >> x;
            st.update(l - 1, r - 1, x);
        } else {
            cout << st.query(l - 1, r - 1) << '\n';
        }
```

```
    }
}
```

## 8.2 Range assign + range add + range sum (CSES 1735)

Tested on the CSES sample (output `7 11 15`) and against a brute force on 400 random cases.

```
#include <bits/stdc++.h>
using namespace std;

// CSES 1735 - Range Updates and Sums.
//   1 a b x : add x to a[a..b]
//   2 a b x : set a[a..b] = x
//   3 a b   : print sum of a[a..b]
// Tag = (assign?, setVal, add): "first (optionally) set everything to setVal, then add
`add`".
struct Tag {
    bool hasSet = false;
    long long setVal = 0, add = 0;
};

int n;
vector<long long> sum;
vector<Tag> tag;

void applyTag(int x, int len, const Tag& t) {
    if (t.hasSet) {                 // assignment wipes out everything below
        sum[x] = t.setVal * len;
        tag[x].hasSet = true;
        tag[x].setVal = t.setVal;
        tag[x].add = 0;
    }
    if (t.add != 0) {               // addition stacks on top of whatever is there
        sum[x] += t.add * len;
        tag[x].add += t.add;
    }
}

void push(int x, int l, int r) {
    int m = (l + r) / 2;
    applyTag(2 * x, m - l + 1, tag[x]);
    applyTag(2 * x + 1, r - m, tag[x]);
    tag[x] = Tag();
}

void build(int x, int l, int r, const vector<long long>& a) {
    if (l == r) { sum[x] = a[l]; return; }
    int m = (l + r) / 2;
    build(2 * x, l, m, a);
    build(2 * x + 1, m + 1, r, a);
    sum[x] = sum[2 * x] + sum[2 * x + 1];
}
```

```cpp
void update(int x, int l, int r, int ql, int qr, const Tag& t) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) { applyTag(x, r - l + 1, t); return; }
    push(x, l, r);
    int m = (l + r) / 2;
    update(2 * x, l, m, ql, qr, t);
    update(2 * x + 1, m + 1, r, ql, qr, t);
    sum[x] = sum[2 * x] + sum[2 * x + 1];
}

long long query(int x, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return sum[x];
    push(x, l, r);
    int m = (l + r) / 2;
    return query(2 * x, l, m, ql, qr) + query(2 * x + 1, m + 1, r, ql, qr);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto& v : a) cin >> v;
    sum.assign(4 * n, 0);
    tag.assign(4 * n, Tag());
    build(1, 0, n - 1, a);
    while (q--) {
        int type, l, r;
        cin >> type >> l >> r;
        --l; --r;
        if (type == 1) {
            Tag t; cin >> t.add;
            update(1, 0, n - 1, l, r, t);
        } else if (type == 2) {
            Tag t; t.hasSet = true; cin >> t.setVal;
            update(1, 0, n - 1, l, r, t);
        } else {
            cout << query(1, 0, n - 1, l, r) << '\n';
        }
    }
}
```

## 9. Common mistakes

- **Not pushing in `query`.** The query reads children, so it must push first, exactly like `update`.
- **Not pulling after `update`.** The parent's value goes stale.
- **Forgetting the segment length** when applying add/assign to a sum node.
- **Bad "no tag" encoding.** `assign = 0` as "no tag" breaks when the problem assigns 0. Use a flag.

- **Wrong composition order.** "Add then set" is "set"; "set then add" is "set then add". Write the table before coding.
- **Pushing from a leaf** with tightly sized arrays. In the recursive structure above, leaves are never pushed because a leaf is always either disjoint or covered.
- **Overflow.** `x·len` can be 10^9·2·10^5 = 2·10^14; sums of such values need `long long` (and for repeated additions, check that the problem's limits keep sums within 9·10^18).
- **Mixing 0- and 1-indexed** ranges between input and tree.
- **Using a lazy tree when a BIT suffices** — not a bug, but costs time in contest.

## 10. Practice problems

Warm-up:

- CSES 1651 — *Range Update Queries* — https://cses.fi/problemset/task/1651
- Codeforces 52C — *Circular RMQ* — https://codeforces.com/problemset/problem/52/C

Core:

- CSES 1735 — *Range Updates and Sums* — https://cses.fi/problemset/task/1735
- CSES 1736 — *Polynomial Queries* — https://cses.fi/problemset/task/1736
- Codeforces 242E — *XOR on Segment* — https://codeforces.com/problemset/problem/242/E
- Codeforces EDU "Segment Tree, part 2" — https://codeforces.com/edu/course/2/lesson/5

Challenge:

- Codeforces 145E — *Lucky Queries* — https://codeforces.com/problemset/problem/145/E
- Codeforces 620E — *New Year Tree* — https://codeforces.com/problemset/problem/620/E
- Codeforces 343D — *Water Tree* — https://codeforces.com/problemset/problem/343/D
- Codeforces 558E — *A Simple Task* — https://codeforces.com/problemset/problem/558/E
- Library Checker — *Range Affine Range Sum* — https://judge.yosupo.jp/problem/range_affine_range_sum

## 11. Further reading

- CP-Algorithms, "Segment Tree", section "Range updates (Lazy Propagation)" — https://cp-algorithms.com/data_structures/segment_tree.html
- AtCoder Library `lazy_segtree` documentation — the monoid/mapping/composition interface of Section 6 in library form — https://atcoder.github.io/ac-library/production/document_en/lazysegtree.html
- USACO Guide, "Range Update Range Query" — https://usaco.guide/plat/RURQ
- Next week: Week 26 returns to dynamic programming with bitmask DP. Week 34 (sqrt decomposition) gives an alternative when no compact lazy tag exists.